INDUSTRIAL INTERNSHIP REPORT

Air Pollution Monitoring System

An ESP32-Based IoT Solution for Air-Quality Indication and Environmental Monitoring

PREPARED BY	Giri G
DOMAIN	Embedded Systems & IoT
ORGANIZED BY	upskill Campus (USC) & The IoT Academy
INDUSTRY PARTNER	UniConverge Technologies Pvt Ltd (UCT)
SUBMISSION DATE	17/10/2026

Project report template adapted to the supplied Smart Home Automation report structure.

EXECUTIVE SUMMARY

This report describes the proposed design of an Air Pollution Monitoring System built around an ESP32 microcontroller, an MQ-135 broad gas/air-quality sensor, and a DHT11 temperature and humidity sensor. The ESP32 reads sensor values, reports them over Wi-Fi using MQTT, and provides a configurable local buzzer alert when the raw MQ-135 reading crosses a prototype threshold.

The system is intended as an educational IoT prototype for observing changes in sensor response and environmental conditions. MQ-135 raw ADC readings are not a certified AQI or direct ppm measurement; calibration and appropriate validation are required before quantitative interpretation.

TABLE OF CONTENTS

Section	Title
1	Preface
2	Introduction
2.1–2.3	Project objectives, references and glossary
3	Problem Statement
4	Existing and Proposed Solution
5	Proposed Design / Model
6	Performance Test Plan
7	My Learnings
8	Future Work Scope
9	Source Code Implementation

1. Preface

This report documents an Embedded Systems & IoT project concept prepared under the internship framework associated with upskill Campus (USC), The IoT Academy, and UniConverge Technologies Pvt Ltd (UCT). The work applies sensor interfacing, microcontroller firmware, Wi-Fi connectivity, MQTT messaging, and technical documentation to an environmental-monitoring use case.

The system is designed as a prototype that can be assembled and tested with commonly available components. Its readings should be interpreted as indicative unless the sensors are calibrated and compared against a trusted reference instrument.

2. Introduction

Air pollution can vary with traffic, industrial activity, cooking, ventilation, and other local conditions. A low-cost sensor node can help learners observe changes over time and view readings remotely. The proposed system combines an MQ-135 sensor response with temperature and humidity context and sends the values to an MQTT broker for display in a compatible dashboard.

2.1 Project Objectives

- Build an ESP32-based sensor node for air-quality indication and environmental monitoring.
- Read MQ-135 analog output and DHT11 temperature/humidity values.
- Publish readings and alert status using MQTT over Wi-Fi.
- Provide an optional local buzzer alert when a configurable raw sensor threshold is exceeded.
- Document wiring, firmware setup, test procedures, limitations, and future improvements.

2.2 References

- Espressif Systems, ESP32 Series Datasheet and Technical Reference Documentation.
- MQ-135 gas sensor module datasheet/application documentation from the selected module manufacturer.
- Adafruit DHT sensor library documentation.
- OASIS, MQTT Version 3.1.1 Specification.

2.3 Glossary

Term	Meaning
IoT	Internet of Things
ESP32	Microcontroller with integrated Wi-Fi and Bluetooth
MQ-135	Broad gas sensor commonly used for qualitative air-quality indication
ADC	Analog-to-Digital Converter
MQTT	Lightweight publish/subscribe messaging protocol
GPIO	General Purpose Input/Output
Telemetry	Sensor data sent from a device to another system

3. Problem Statement

Many locations lack continuous, low-cost visibility into changing air-quality conditions. Manual observation cannot provide regular measurements or remote access to readings. The project requirement is to develop an accessible prototype that samples a gas-sensor response, records basic environmental conditions, transmits data remotely, and indicates when a configured sensor threshold is exceeded.

The prototype does not replace regulatory monitoring stations or certified safety detectors. Sensor response depends on warm-up, calibration, temperature, humidity, and the gases present.

4. Existing and Proposed Solution

Existing approaches and limitations

- Visual/manual observation: does not provide continuous numeric records.
- Standalone indicators: may not provide remote telemetry or historical data.
- Professional monitoring equipment: can offer higher accuracy but may be costly for a small educational prototype.

Proposed solution

An ESP32 reads the MQ-135 analog signal and DHT11 readings. It publishes the raw sensor value, temperature, humidity, and a threshold-based status over MQTT. An optional buzzer provides local indication when the raw reading exceeds the configured example threshold. A dashboard or MQTT client can subscribe to the topics to display readings.

5. Proposed Design / Model

5.1 High-Level Architecture (HLD)

SENSING LAYER	PROCESSING LAYER	CONNECTIVITY	PRESENTATION
MQ-135 DHT11	ESP32 ADC + firmware	Wi-Fi MQTT broker	Dashboard / MQTT client Alert status
Sensor inputs	Read, compare, format	Publish telemetry	View readings

Figure 5.1: High-level architecture of the proposed system.

5.2 Low-Level Diagram (LLD) and Circuit Interfacing

Component	ESP32 connection	Purpose / note
MQ-135 module AO	GPIO 34 (ADC input)	Raw analog sensor response
DHT11 DATA	GPIO 15	Temperature and humidity
Optional active buzzer	GPIO 18	Threshold alert; use a driver if needed
Power and ground	As specified by module	Common ground; check voltage compatibility

Electrical caution: ESP32 GPIO pins are not 5-V tolerant. Verify the MQ-135 module's analog output voltage and add an appropriate divider or level shifter if required. The exact wiring depends on the purchased module.

5.3 System Execution Flow

1	Power ON
2	Initialize GPIO, ADC and DHT11
3	Connect to Wi-Fi
4	Connect to MQTT broker
5	Read MQ-135 ADC + DHT11 values
6	Compare raw ADC value with configured threshold
7	Publish telemetry and set buzzer status as applicable
8	Repeat every 5 seconds / maintain MQTT connection

6. Performance Test Plan

The following test cases are a validation plan, not a claim that tests have already passed. Record observed results after running the hardware.

Test ID	Scenario	Procedure	Expected result	Status
TC-01	Sensor initialization	Power the ESP32 and open Serial Monitor at 115200 baud.	MQ-135 ADC values appear; DHT11 values appear when sensor is connected.	To test
TC-02	MQTT publishing	Subscribe to project MQTT topics and run the device.	Raw ADC, temperature, humidity, and status messages are received.	To test
TC-03	Threshold alert	Set a test threshold and observe readings under different conditions safely.	Status changes when raw ADC crosses the configured threshold.	To test
TC-04	Wi-Fi/MQTT recovery	Interrupt and restore Wi-Fi or broker access.	Device attempts reconnection and resumes publishing.	To test
TC-05	DHT11 read error	Disconnect the DHT11 data line while powered down, then retest.	Firmware reports failed reading instead of publishing invalid temperature/humidity.	To test

6.1 Performance Outcomes and Constraints

- Sampling interval: configured to publish every 5 seconds; actual timing depends on Wi-Fi and MQTT behavior.
- Sensor interpretation: raw MQ-135 ADC values are relative readings and require calibration for meaningful concentration estimates.
- Connectivity: public MQTT broker access is useful for prototyping but is not a secure private production channel.
- Reliability: warm-up, power stability, wiring, sensor placement, and ambient conditions can affect readings.

7. My Learnings

- Sensor interfacing: reading analog sensor signals and digital temperature/humidity data.
- Embedded firmware: initializing GPIO, reading sensors, and scheduling periodic tasks.
- IoT communication: configuring Wi-Fi and MQTT publish topics for remote telemetry.
- Testing and documentation: separating expected outcomes from results that have actually been observed.

8. Future Work Scope

- Add a calibrated particulate-matter sensor such as an appropriate PM2.5 sensor for particle measurements.
- Create a dashboard with trend graphs, timestamped history, and notifications.
- Calibrate the MQ-135 response against a trusted reference instrument and document environmental conditions.
- Use authenticated MQTT over TLS and unique credentials for deployment.
- Enclose the electronics safely and validate sensor placement and power supply design.

9. Source Code Implementation

The following sketch is included as a separate source-code file and in the accompanying source-code submission PDF. Update the Wi-Fi credentials before uploading. The example threshold is configurable and should be tuned only after observing and calibrating the sensor.

MQTT Topics

- airpollution/mq135_raw_adc
- airpollution/temperature_c
- airpollution/humidity_percent
- airpollution/alert

Software prerequisites

- Arduino IDE with ESP32 board support.
- PubSubClient library.
- Adafruit DHT sensor library and Adafruit Unified Sensor dependency if prompted.

The complete firmware is provided in AirPollutionMonitoring.ino and reproduced in the source-code submission PDF. A public broker is used only for prototype demonstration; do not treat it as a secure deployment configuration.

END OF REPORT